PWA "Mi Billetera Lite"

Objetivo General: Desarrollar una Aplicación Web Progresiva (PWA) funcional enfocada en el registro de gastos diarios, implementando capacidades de funcionamiento sin conexión (offline), persistencia de datos local e instalabilidad en dispositivos móviles y de escritorio.

1. Enunciado del Proyecto

Contexto para el alumno: Imagina que un cliente necesita una aplicación ultraligera para registrar sus gastos sobre la marcha (viajes en transporte público, compras en mercados, pagos en zonas rurales). El problema principal es que en muchas de estas zonas no hay cobertura de datos móviles.

Tu misión es construir **"Mi Billetera Lite"**, una PWA que permita al usuario registrar la cantidad, el concepto y la categoría de un gasto. La aplicación debe cargar instantáneamente, ser instalable como una app nativa desde el navegador y, lo más importante, **debe funcionar al 100% aunque el dispositivo esté en modo avión**.

2. Requerimientos Funcionales

El usuario final debe ser capaz de interactuar con la siguiente funcionalidad en la interfaz:

- **Formulario de Registro:** Un formulario simple para agregar un gasto. Debe contener:
  - *Concepto* (Texto, ej: "Café").
  - *Monto* (Numérico, ej: 45.50).
  - *Categoría* (Selector: Comida, Transporte, Servicios, Ocio, Otros).
  - *Botón* para guardar.
- **Historial de Gastos:** Una lista o tabla debajo del formulario que muestre todos los gastos registrados, ordenados del más reciente al más antiguo.
- **Totalizador:** Un indicador visual en la parte superior que sume el total de los gastos registrados en la lista.
- **Diseño Responsivo:** La interfaz debe estar optimizada para verse y usarse cómodamente en pantallas de teléfonos móviles (Mobile First).

3. Requerimientos Técnicos (Arquitectura PWA)

Para que el proyecto sea considerado una PWA válida, el alumno debe implementar:

- **Web App Manifest (`manifest.json`):**
  - Definir `name`, `short_name`, `theme_color`, `background_color`, y `display: "standalone"`.
  - Incluir al menos dos tamaños de íconos (ej. 192x192 y 512x512) compatibles con máscaras (*maskable*).
- **Service Worker (`sw.js`):**

    - Fase de `install`: Cachear los archivos estáticos necesarios para renderizar la interfaz gráfica (App Shell).
    - Fase de `activate`: Limpiar cachés antiguas si hay una nueva versión del Service Worker.
    - Fase de `fetch`: Implementar una estrategia **Cache First, Network Fallback** (servir desde caché si existe, si no, buscar en la red) para los archivos del App Shell.
- **Persistencia de Datos:**
    - Los gastos no se enviarán a un servidor externo. Deben guardarse en el navegador utilizando `LocalStorage` (nivel básico) o `IndexedDB` (nivel avanzado/recomendado).
- **Seguridad:**
    - La aplicación debe servirse obligatoriamente a través de HTTPS (o `localhost` para el entorno de desarrollo).

4. Criterios de Aceptación (Checklist de Evaluación)

El proyecto se dará por aprobado si cumple los siguientes escenarios de prueba durante la revisión:

1. **Prueba de Instalación:**
    - *Acción:* Abrir la app en Chrome (desktop o Android).
    - *Resultado esperado:* El navegador debe mostrar el prompt/icono para "Instalar aplicación". Al instalarla, debe abrirse en su propia ventana sin barra de direcciones (standalone).
2. **Prueba de Auditoría Lighthouse:**
    - *Acción:* Correr el reporte "PWA" en Chrome DevTools.
    - *Resultado esperado:* Debe aparecer el check verde o el badge de que cumple con los requerimientos base de PWA (Manifest válido, Service Worker registrado).
3. **Prueba de Supervivencia Offline:**
    - *Acción:* En Chrome DevTools > Pestaña *Network*, cambiar el throttling a **Offline**. Recargar la página (F5).
    - *Resultado esperado:* La página debe cargar perfectamente con sus estilos CSS y lógica JS, sin mostrar el dinosaurio de Chrome.
4. **Prueba de Persistencia y Operación sin Red:**
    - *Acción:* Estando en modo **Offline**, el usuario agrega un gasto ("Taxis", $150). Cierra la pestaña, la vuelve a abrir (siguiendo offline).
    - *Resultado esperado:* El gasto de $150 sigue en el historial y el total refleja la suma correcta.

5. Entregables Esperados

- Repositorio en GitHub con el código fuente (HTML, CSS, JS, manifest, íconos).
- URL de la aplicación desplegada de forma gratuita con HTTPS (se sugiere GitHub Pages, Vercel, Netlify o Firebase Hosting).